

CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 12 · LECTURE NOTES

Cloud, Scaling, and Economics

What it costs, and what the bill is actually for.

MODULE OUTCOMES

- Explain elasticity and what it is worth.
- Compare the compute abstractions and their trade-offs.
- Explain data gravity and egress economics.
- Explain autoscaling and why it is harder than it looks.
- Decide honestly between cloud and owned hardware.

1 What you are actually buying

THE CLOUD SELLS ELASTICITY, AND IT IS EXPENSIVE PER UNIT

Rented compute costs several times what owned compute costs when amortised over a three-year hardware life. This is not a secret, the providers do not dispute it, and it is not the point of the product.

What you are buying is the ability to have zero servers on Monday and four hundred on Friday, provisioned in minutes, and to pay for neither at any other time. You are buying the removal of a capital decision and a lead time.

So the cloud wins when load is variable, unpredictable, or growing — a product launch, a seasonal peak, an experiment that may be cancelled, a startup that may not exist in a year. **Optionality has real value and the premium buys it.**

And it loses when load is steady and known. A game with a stable player count running continuously is the textbook case for owned hardware — and several large companies have repatriated workloads for exactly this reason, which is worth knowing because the default assumption now runs the other way.

Abstraction	What you manage	What it suits
Bare metal	Everything, including the hardware.	Steady predictable load, latency-critical work, and GPU workloads where virtualisation overhead or availability is a problem.
Virtual machines	The operating system and everything above.	The flexible default. Full control, and you own patching, monitoring, and configuration.

Abstraction	What you manage	What it suits
Containers	The image; a scheduler decides where it runs.	Most stateless services. Kubernetes if the scale genuinely warrants it — it is a substantial operational commitment and is frequently adopted well before it is needed.
Serverless functions	A function and its dependencies.	Spiky, stateless, short-lived work. Cold starts make it unsuitable for latency-sensitive paths, and the per-invocation pricing becomes expensive at sustained high volume.
Managed services	Configuration only.	Databases, queues, caches, object storage. Usually the right choice — operating a database correctly across backups, failover, upgrades, and replication is a genuine specialism, and the price premium is usually smaller than the cost of learning it badly.

2 Where the bill comes from

EGRESS AND DATA GRAVITY

Ingress is typically free and egress is charged, at rates that add up far faster than people expect. **A game streaming state to a hundred thousand concurrent clients pays per byte, continuously**, and that line can exceed the compute bill — which makes the bandwidth optimisations of Module 09 §4 directly financial rather than merely technical.

And cross-zone and cross-region traffic is charged *inside* the cloud too. A chatty microservice architecture spread across availability zones for resilience pays for every internal call that crosses a zone boundary — **so an architectural choice made for availability has a continuous and largely invisible cost**, and it is usually discovered during a cost review rather than a design review.

Data gravity: once a petabyte lives in one provider, moving it costs more than leaving it. The egress charge on the full dataset may exceed a year of storage, so the data stays and the compute that uses it must stay with it. **The egress charge is the lock-in mechanism**, whether or not it was designed as one.

So compute is portable and data is not. Architect with that asymmetry in mind, and measure egress from the first week — it is the line nobody budgets for and the one that produces surprises.

3 Autoscaling

AUTOSCALING REACTS TO A SIGNAL THAT LAGS THE PROBLEM

By the time CPU utilisation is high, users are already waiting — and a new instance takes anywhere from thirty seconds to several minutes to boot, warm up, pass health checks, and begin serving.

So scaling on a lagging metric is structurally always late. Scaling on a leading indicator — queue depth, request arrival rate, connection count — is better, and requires understanding the relationship between that indicator and the capacity needed, which has to be measured rather than guessed.

And it can oscillate. Scale up, per-node load falls below the scale-down threshold, scale down, load rises, scale up again. **Hysteresis and cooldown periods exist precisely to prevent this** and they have to be tuned against the actual startup time — which is the same flapping problem as Module 07 §4, in a different domain.

Worse, scaling can deepen the incident it was meant to relieve. New instances start with cold caches and serve slowly; they all open database connections simultaneously and exhaust the connection pool; they all fetch the same configuration at once. **A scaling event during an overload can be the thing that turns a degradation into an outage**, which is why scale-up needs to be rate-limited as well as fast.

- **Scale on a leading indicator** — queue depth or request rate rather than CPU — and validate the relationship with a load test rather than assuming it.
- **Pre-warm for predictable peaks.** A game's evening peak is known to the hour, so scale on the clock rather than waiting for the symptom. Scheduled scaling is unglamorous and is usually better than reactive scaling.
- **Scale up fast and down slow.** The costs are deeply asymmetric: **being too small is an outage and being too large is a line item**, so the thresholds should not be symmetric either.
- **Set a maximum.** A runaway autoscaler responding to a bug — **an infinite retry loop, a queue that never drains** — produces a very large bill very quickly, and this has happened to a great many people. **The maximum is the control people skip** and then learn about from their invoice.
- **And load-shed rather than collapse.** **Serving 90% of users well beats serving 100% of them badly or none of them at all**, and that policy has to be designed in — a system that has no shedding mechanism will simply fall over.

4 Game servers specifically

Property	Consequence
Sessions are stateful and long-lived.	A match cannot be moved mid-game, so ordinary load balancing does not apply — you place a session at start and live with the decision. Draining a node means waiting for matches to end.

Property	Consequence
Latency-bound.	Servers must be physically near players , so you need many regions each running modest capacity — which is the opposite of the consolidation that makes capacity planning easy.
Sharply peaked demand.	Evenings and weekends run five to ten times the trough , and the peak moves around the world with the timezone — which is either an opportunity (follow the sun with the same capacity) or a complication, depending on your architecture.
Launch spikes.	Day one can be fifty times month two . Provisioning for launch and owning that hardware afterwards is wasteful; this is the clearest case for cloud bursting there is.
Long-lived processes.	Serverless does not fit at all — the model assumes short stateless invocations and a game server is neither.
CPU-bound simulation.	Core count and single-thread performance matter far more than memory or storage, which changes the instance selection.

DECIDING HONESTLY

What is the ratio of your peak to your trough? Below about 2:1, owned hardware probably wins on cost. Above about 10:1, the cloud probably does. **Compute that number before having the argument**, because it settles most of it.

How predictable is the peak? A predictable peak can be pre-provisioned on either platform; an unpredictable one is exactly what elasticity is for, and is where the premium is worth paying.

How much data leaves? **At scale, egress can exceed the compute bill entirely** (§2), and it is the term most likely to be missing from the comparison spreadsheet.

And what is your team's time worth? **Managed services buy back operational attention** — the hours not spent on backups, failover testing, and patching — and **for a small team that is usually the dominant term in the whole calculation**. It is also the argument least often made explicitly, because it is the hardest to put a number on, which is not a reason to leave it out.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Google — Site Reliability Engineering, chapters on capacity and load (free)	The §3 material , including load shedding and the cascading-failure patterns that scaling can trigger.
Armbrust et al. — A View of Cloud Computing (free)	The §1 economics , argued carefully. Old, and the argument is unchanged.

Resource	What it covers for this module
37signals — the cloud repatriation writeups (free)	The counter-case with real numbers. Read alongside the Berkeley paper rather than instead of it.
Agones, and the major providers' game-server hosting documentation (free)	The \$4 workload, handled by tooling built specifically for stateful session placement.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Price your Module 09 project** on owned hardware and on three cloud providers, at your expected scale.
- 2 **Compute the egress cost** from your measured bandwidth per client per second, at 1,000 and at 100,000 concurrent players.
- 3 Measure the cost of the bandwidth optimisations of Module 09 in currency rather than bytes.
- 4 Deploy a service and **measure cold start time** for VMs, containers, and serverless.
- 5 **Implement autoscaling on CPU**, then on queue depth, and compare how late each reacts to a load spike.
- 6 **Make your autoscaler oscillate**, then fix it with hysteresis and a cooldown.
- 7 Simulate a scale-up during overload where new instances exhaust the database connection pool.
- 8 Implement load shedding and show it keeps 90% of users served under an overload that otherwise collapses.
- 9 **Compute your peak-to-trough ratio** for a real or projected workload.
- 10 **Write the honest comparison** for your project, including the team-time term.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does the cloud actually sell, and when does it lose?
- 2 Compare five compute abstractions and what each suits.
- 3 Why do egress and cross-zone traffic dominate bills?
- 4 What is data gravity and how does it create lock-in?
- 5 Why is autoscaling structurally late, and how can it oscillate?
- 6 How can scaling deepen an incident?
- 7 Give five autoscaling practices and say which is most often skipped.
- 8 Name six ways game hosting differs, and the usual conclusion.

9 What four questions decide cloud against owned?